

run1-5-docker.sh

You need to understand how run1-5-docker.sh works to make your own script.

Learning Goals

- Explain the ten steps of `run1-5-docker.sh` and how they differ from `run1-5.sh`.
- Distinguish `docker compose run` (new container) from `docker compose exec` (running container).
- Read the script well enough to adapt it for your own project.

What run1-5-docker.sh Sets Up

1. Build: Build the nginx, php, and mysql images
2. Prepare: Create the Laravel project (only once) in a php container
3. Configure: Point Laravel's `.env` at `studentdb`
4. Install: Composer packages and the app key
5. Sanctum: Run `install:api`
6. Copy: Copy the lesson files into the project
7. Start: Start the three containers and wait until they are healthy
8. Migrate: Create the `students`, `users`, and token tables if missing
9. Seed: Insert sample students and two demo users

`run1-5-docker.sh` follows the same steps as `run1-5.sh` .

- Sanctum, copying files, migrating, and seeding (Steps 5, 6, 8, 9) are the same as in Module 2. See `5-2. run1-5.sh.md` .
- The new parts are Steps 1 to 4, 7, and 10: everything runs in containers.

```
cd "4. Containerizing Laravel with Docker"  
bash ./run1-5-docker.sh
```

Only Docker is needed. The first run builds three images, so it takes a few minutes.

What Changes from `run1-5.sh`

<code>run1-5.sh</code>	<code>run1-5-docker.sh</code>
<code>composer</code> and <code>php</code> on your computer	A throw-away <code>php</code> container (<code>php_run</code>)
Is MySQL running?	<code>up -d --wait</code> waits for the health check
<code>sudo mysql</code> : create database and user	<code>.env</code> creates them when the volume is new
<code>sed</code> edits the project's <code>.env</code>	Copy <code>.env.docker</code> to <code>student-api/.env</code>
<code>php artisan serve --port=8080</code>	Nginx container on port 8000

Type It Yourself First

The script only saves typing. To learn, run the key commands once by hand.

```
docker compose build
docker compose run --rm --no-deps php composer --version
docker compose up -d --wait
docker compose ps
docker compose exec php php -v
docker compose down
```

- Read each output. Then read the script to see where the same commands appear.
- A script you cannot read is a script you cannot fix.

Step 1: The `php_run` Helper

```
php_run() { docker compose run --rm --no-deps -T -e COMPOSER_ALLOW_SUPERUSER=1 php "$@"; }
```

Part	Meaning
<code>docker compose run</code>	Start a new container from the <code>php</code> service
<code>--rm</code>	Delete that container when the command ends
<code>--no-deps</code>	Do not start <code>mysql</code> or <code>nginx</code> for this command
<code>-T</code>	No terminal: works in scripts
<code>-e</code>	Composer may run as root

Step 2: Create the Project

```
docker compose run --rm --no-deps -T -e COMPOSER_ALLOW_SUPERUSER=1 -v "$SCRIPT_DIR:/work" -w /work php \
composer create-project laravel/laravel student-api "^13.0" --no-scripts
```

- `-v "$SCRIPT_DIR:/work"` mounts this folder inside the container, so `student-api/` appears on your computer.
- `-w /work` sets the working directory.
- `"^13.0"` asks for Laravel 13. The script skips this step if `student-api/artisan` exists.
- `--no-scripts` skips Laravel's default setup (SQLite, key). We configure it ourselves.

Step 3: Configure the Database

```
if [[ ! -f "$TARGET_DIR/.env" ]]; then
    docker compose run --rm --no-deps -T -v "$SCRIPT_DIR/.env.docker:/tmp/env.docker:ro" php \
        cp /tmp/env.docker .env
fi
```

- Module 2 edited `.env` with `sed`. Here the whole file is ready: `.env.docker`.
- `:ro` mounts it read-only.
- The copy runs **inside** the container, because the project folder was created there. On Linux/WSL2 your user may not own it.

Step 4: Install and Generate the Key

```
php_run composer install --no-interaction --no-scripts
php_run php artisan package:discover
if ! grep -q '^APP_KEY=base64:' "$TARGET_DIR/.env"; then
    php_run php artisan key:generate --force
fi
```

- `composer install` makes sure `vendor/` is complete.
- `package:discover` registers installed packages, because `--no-scripts` skipped it.
- `key:generate` runs only when `APP_KEY` is empty, so a rerun keeps your key.

Step 5: Sanctum

```
if [[ ! -f "$TARGET_DIR/config/sanctum.php" ]]; then
    php_run php artisan install:api --no-interaction --without-migration-prompt
fi
```

- `install:api` installs Laravel Sanctum and creates the `personal_access_tokens` migration file.
- It is the same command as in Module 2 (5-1), now run in the `php` container.
- `--without-migration-prompt` : do not ask to run migrations. The script runs them later.
- If `config/sanctum.php` exists, Sanctum is already installed.

Step 6: Copy the Lesson Files

File	Role
<code>Api/AuthController.php</code>	<code>login</code> , <code>me</code> , <code>logout</code>
<code>Api/StudentController.php</code>	Student actions (reads are public)
<code>Models/Student.php</code>	Same model as Lesson 4
<code>Models/User.php</code>	Generated model plus the <code>HasApiTokens</code> trait

File	Ro
<code>routes/api.php</code>	Public ro and the <code>auth:sa</code> group
<code>2024_01_01_000000_create_students_table.php</code>	Same tal definition Lesson 4
<code>StudentSeeder.php</code> , <code>UserSeeder.php</code>	Sample students demo us

- These are the same eight files as in Module 2, taken from `student-api1-5/` .

```
docker compose run --rm --no-deps -T -v "$SOURCE_DIR:/source:ro" php sh -c '
    mkdir -p app/Http/Controllers/Api
    for f in \
        app/Http/Controllers/Api/AuthController.php \
        ...
    do
        cp "/source/$f" "$f"
    done'
```

- `student-api1-5/` is mounted read-only at `/source`. The loop copies each file into the project.
- The copy runs in the container for the same ownership reason as Step 3.

Permissions

```
php_run chown -R www-data:www-data storage bootstrap/cache
```

- PHP-FPM runs as `www-data` . Laravel writes logs, sessions, and caches into these two folders.
- The bind mount hides the permissions of the image, so the script sets them.
- Without it, Linux/WSL2 shows "Permission denied" in the browser.

Step 7: Start the Three Containers

```
docker compose up -d --wait  
docker compose exec -T php php artisan config:clear
```

Part	Meaning
<code>up</code>	Create and start the services
<code>-d</code>	In the background
<code>--wait</code>	Return only when every service with a health check is healthy
<code>exec -T</code> <code>php</code>	Run a command in the running <code>php</code> container

- `run` creates a new throw-away container. `exec` uses a container that is already running.

Health Checks

Service	Health check
mysql	mysqladmin ping as root
php	php-fpm -t (configuration test)
nginx	wget the home page on 127.0.0.1

- nginx:alpine has wget but no curl , so the check uses wget .
- If a service never becomes healthy, up --wait fails and the script stops.
- Find out why: docker compose logs mysql php nginx .

Step 8: Skip a Table That Already Exists

`install:api` names the token migration with **today's** timestamp.

- You delete `student-api/` and run the script again. The MySQL volume still has `personal_access_tokens`.
- Laravel sees a new migration file, tries to create the table again, and fails with `1050 Table already exists`.

```
table_exists() {  
  docker compose exec -T -e T="$1" mysql sh -c \  
    'mysql -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" -N -e "SHOW TABLES LIKE \"\$T\"" "$MYSQL_DATABASE" 2>/dev/null' | grep -q .  
}
```

Syntax	Meaning
<code>exec -T -e</code> <code>T="\$1" mysql</code>	Run in the <code>mysql</code> container; pass the table name as <code>T</code>
<code>sh -c '...'</code>	The single quotes keep <code>\$MYSQL_USER</code> for the container to read
<code>SHOW TABLES</code> <code>LIKE "users"</code>	Prints the table name if it exists, nothing if not
<code>-N</code>	Do not print the column header
<code>grep -q .</code>	Succeeds if there was any output at all

- The container already has `MYSQL_USER`, `MYSQL_PASSWORD`, and `MYSQL_DATABASE` from `.env`. The script never types a password.

```
table_exists users || docker compose exec -T php php artisan migrate --path=... --force  
table_exists personal_access_tokens || docker compose exec -T php php artisan migrate --path="$TOKENS_MIGRATION" --force
```

- `a || b` runs `b` only if `a` **fails**: here, only if the table is missing.
- The students migration has its own `Schema::hasTable()` check, so it always runs.
- We run migrations one file at a time with `--path`, as in Module 2.

Step 9: Seed and Verify

```
docker compose exec -T php php artisan db:seed --class=StudentSeeder --force
docker compose exec -T php php artisan db:seed --class=UserSeeder --force
docker compose exec -T php php artisan route:list --path=api
```

- `UserSeeder` uses `User::firstOrCreate()`, so rerunning does not add duplicate users.

Email	Password
student@university.edu	student123
teacher@university.edu	teacher456

Step 10: A Quick Check

```
if command -v curl >/dev/null 2>&1; then
    curl -fsS -H 'Accept: application/json' http://localhost:8000/api/students/1 >/dev/null \
        && echo "Public route OK: http://localhost:8000/api/students/1"
fi
docker compose ps
```

- The request goes through all three containers: Nginx → PHP-FPM → MySQL.
- `curl -f` fails on an HTTP error, and `set -e` then stops the script.
- `docker compose ps` shows the three services. Next lesson: `4-3`, testing.

Command Summary

Students should remember:

Command	Meaning
<code>docker compose run --rm --no-deps php ...</code>	One-off command in a new <code>php</code> container
<code>docker compose exec -T php php artisan ...</code>	Command in the running <code>php</code> container
<code>docker compose up -d --wait</code>	Start everything and wait until healthy

Command	Meaning
<code>docker compose exec -T mysql mysql ...</code>	Query the database in its container
<code>php artisan install:api</code>	Install Sanctum
<code>php artisan migrate --path=... --force</code>	Run one migration file
<code>php artisan db:seed --class=... --force</code>	Run one seeder